

DV Interview Cheat-Sheet

50 free interview questions with real answers

SystemVerilog basics, UVM, SVA, and functional coverage -- written by a working DV/UVM engineer.

What's inside:

- SV Basics (12 questions)
- UVM (18 questions)
- SVA (10 questions)
- Coverage (10 questions)

Want more? The full interactive version -- filterable, searchable -- is free at <https://dv-verification-hub.vercel.app/tools/interview-questions>

SV Basics

What's the difference between a logic and a wire/reg in SystemVerilog?

`logic` is a single 4-state data type that replaces the need to choose between `wire` and `reg` for most uses -- it can be driven by a continuous assignment, a procedural block, or a port, as long as it's only ever driven from one place. The one thing `logic` cannot do that `wire` can is be driven by multiple continuous drivers (e.g., a tri-state bus with multiple drivers) -- that still requires an actual `wire`tri`` type.

What is the difference between blocking (`=`) and non-blocking (`<=`) assignments?

Blocking assignments execute immediately and in program order within a procedural block, blocking subsequent statements until they complete -- used for combinational logic (``always_comb``). Non-blocking assignments schedule the update to happen at the end of the current time step (after the NBA region), letting all right-hand sides in a clocked block be evaluated using pre-update values -- this is why non-blocking assignments are mandatory for correctly modeling sequential logic (``always_ff``).

What's the difference between an interface and a class in SystemVerilog?

An `interface`` bundles signals (wires) for connecting hardware blocks together, and can contain modports, clocking blocks, and even tasks/functions for a testbench-side abstraction (like a virtual interface's ``.master`` protocol tasks). A `class`` is a pure software construct -- it has no signals, only variables, methods, and objects allocated on the heap -- used for testbench data/behavior (transactions, sequences, environment structure), never for RTL connectivity.

Explain the difference between `$display`, `$write`, `$strobe`, and `$monitor`.

``$display`/`$write`` print immediately when executed (display adds a newline, write doesn't). ``$strobe`` prints at the end of the current time step, after all other statements scheduled for that time have executed -- useful for printing final settled values instead of a value mid-update. ``$monitor`` continuously watches its argument list and reprints automatically whenever any of them changes, without needing to be called again.

What is the difference between a packed and unpacked array?

A packed array (``bit [7:0] a``) is stored as a contiguous set of bits and can be treated as a single vector for arithmetic/bit-select operations. An unpacked array (``bit a [8]``) is stored as a collection of separate elements (like a C array) with no guaranteed contiguous bit layout, and supports array methods (``.sum()``, ``.find()``, etc.) that packed arrays don't.

What does the ``unique`` keyword do on a case statement, and why use it?

``unique case`` tells the tool that exactly one branch is expected to match for any given input, and asks it to flag (via a runtime/lint warning) both overlapping matches and no-match cases -- this both documents designer intent and helps synthesis infer parallel (not priority-encoded) logic, which can produce better timing than a plain ``case``.

What is the difference between `fork...join`, `join_any`, and `join_none`?

``fork...join`` waits for all forked processes to complete before continuing. ``join_any`` continues as soon as any one of the forked processes completes, leaving the rest running in the background. ``join_none`` continues immediately without waiting for any of them -- the forked processes run fully in parallel with what follows.

What is a virtual interface, and why can't you just use the interface type directly in a class?

A `virtual interface`` is a handle/pointer to an actual interface instance, usable inside a class (classes are dynamic software objects and cannot directly instantiate or reference static hardware constructs like a real interface). It's how testbench components (drivers, monitors) get access to DUT-connected signals -- typically passed in via ``uvm_config_db`` after being set from the top-level testbench module where the real interface instance lives.

What's the difference between \$cast and a normal assignment for class handles?

A normal assignment between class handles only works if the right-hand type is the same as or a subtype of the left-hand type (upcasting), checked at compile time. ``$cast`` performs a runtime-checked downcast (assigning a base-class handle to a more derived type), returning 0 on failure instead of causing a compile error -- essential in UVM when working with factory-created objects typed generically as ``uvm_object``/``uvm_component``.

What is the difference between a mailbox and a queue for inter-process communication?

A ``mailbox`` is a built-in synchronization primitive with blocking ``put()``/``get()`` methods designed for producer/consumer communication between concurrent processes, and can be bounded (fixed capacity, ``put()`` blocks when full). A ``queue`` is just a dynamic data structure (``push_back``, ``pop_front``, etc.) with no built-in blocking or thread-safety -- using a queue for inter-process handoff requires you to write your own synchronization (e.g., with a semaphore or event).

What does `disable iff` do and why is it almost always used in assertions?

``disable iff (condition)`` disables an assertion/property entirely whenever the condition is true -- typically wired to active-low reset (``disable iff (!rst_n)``). Without it, assertions evaluate during reset when signals are often X or in an undefined/transitional state, producing spurious failures that have nothing to do with actual DUT behavior.

What is the difference between a static and automatic variable in SystemVerilog?

A ``static`` variable (the default for module-level/class-level variables, and the default for variables inside tasks unless declared otherwise) has one persistent storage location shared across all calls/instances. An ``automatic`` variable gets a fresh storage location on every call/invoke -- required for recursive tasks/functions, and generally required inside tasks that might have multiple concurrent invocations (e.g., called from a ``fork`` loop), or the invocations will corrupt each other's local variables.

UVM

What is the UVM factory and why use type_id::create() instead of new()?

The factory lets the actual class constructed for a given request be overridden at runtime (via type or instance overrides), without editing the code that requests it. ``new()`` hardcodes the exact class; ``type_id::create()`` asks the factory what to build, defaulting to the requested type unless something has overridden it. This is what makes UVM environments extensible without modifying existing, verified code.

What's the difference between a uvm_sequence and a uvm_sequence_item?

A ``uvm_sequence_item`` is a single transaction (the data). A ``uvm_sequence`` is the procedural code that generates a stream of sequence items and sends them to a sequencer one at a time via ``start_item()``/``finish_item()``, deciding what to generate next based on prior responses if needed.

Explain the UVM phases and which ones run top-down vs bottom-up.

``build_phase`` runs top-down (parent before children) so config/overrides set by a parent are visible when children build. ``connect_phase``, ``end_of_elaboration``, ``start_of_simulation``, and the post-run phases (``extract``, ``check``, ``report``, ``final``) all run bottom-up, so a parent can safely wire together or inspect children that are already fully built/connected. ``run_phase`` (and its runtime sub-phases) is the only phase group where simulation time actually passes, and runs in parallel across all components.

What is the objection mechanism and what happens if you forget to drop an objection?

Objections are a shared counter per phase; ``raise_objection()`` increments it, ``drop_objection()`` decrements it, and the phase only ends once the count returns to zero (after a drain time). Forgetting to drop an objection leaves the counter above zero forever, so ``run_phase`` never ends -- the simulation hangs until a timeout (``+UVM_TIMEOUT``) or wall-clock limit kills it.

What's the difference between `uvm_config_db` and `uvm_resource_db`?

`uvm_resource_db`` is the low-level, globally-scoped storage engine keyed by string name/type. `uvm_config_db`` is a hierarchy-aware wrapper on top of it that resolves `get()` calls using path specificity and set order, which is what enables per-instance configuration overrides. In practice, almost all environment code should use `config_db``; `resource_db`` is used directly only for introspection or deliberately non-scoped defaults.

What is a virtual sequencer and when do you need one?

A virtual sequencer is a `uvm_sequencer`` that owns no driver connection itself -- it only holds handles to the real sequencers in the environment. It's needed the moment a test scenario requires coordinating stimulus across multiple agents/protocols (e.g., configure over APB, then stream over AXI in a specific order), which a single sequence running on a single real sequencer cannot express.

What's the difference between an active and a passive agent?

An active agent contains a driver and sequencer and actively generates/drives stimulus onto the interface. A passive agent contains only a monitor -- it observes bus activity without driving anything, typically used when another master already drives the bus (e.g., monitoring a slave-side interface, or reusing an agent purely for coverage/checking in an environment where you don't own that side of the bus).

Why does a UVM scoreboard usually use a queue instead of comparing transactions one-to-one as they arrive?

Expected and actual transactions rarely arrive in lockstep due to pipelining/latency differences between a reference model and the real DUT. Buffering expected transactions in a queue lets the scoreboard match each actual transaction against the correct pending expected one whenever it eventually arrives, rather than assuming perfect real-time alignment between the two streams.

What is the difference between `get()` and `peek()` on a sequencer/TLM port?

`get()` retrieves and removes the next available item, consuming it. `peek()` retrieves the next available item without removing it, so a later `get()` will return the same item again -- useful when a component needs to inspect an upcoming item before deciding how to react, without committing to consuming it yet.

What does `uvm_component_utils`` do, and why is it required?

It's a macro that registers the component class with the UVM factory and implements required infrastructure methods (like `get_type_name()` and `create()`) automatically. Without it, `type_id::create()` calls for that class will fail because the factory has no registered type to construct -- this is one of the most common 'silent' UVM setup bugs for new class definitions.

What's the difference between `uvm_component` and `uvm_object`?

`uvm_component`` is for the static testbench hierarchy (agents, drivers, monitors, env, test) -- components are built once, persist for the whole simulation, and participate in phasing. `uvm_object`` is for dynamic, transient data (sequence items, sequences, configuration objects) that gets created and destroyed freely throughout the run and does not participate in the component phasing hierarchy.

How does a type override differ from an instance override in the UVM factory?

A type override (`set_type_override``) replaces a class globally, everywhere the factory is asked to create that type. An instance override (`set_inst_override``) replaces it only for a specific hierarchical instance path, leaving all other instances unaffected. When both could apply to the same creation call, the instance override takes precedence.

What is UVM_FRONTDOOR vs UVM_BACKDOOR access in the register model?

`UVM_FRONTDOOR`` drives a register access through the real bus, via the register adapter and the actual sequencer/driver -- it exercises the DUT's bus interface logic. `UVM_BACKDOOR`` (using an HDL path bound via `add_hdl_path``) writes the RTL register directly with zero bus cycles -- fast, but verifies nothing about the bus-facing register access path itself.

What is a `uvm_reg_adapter` for?

It translates between the generic, protocol-agnostic `uvm_reg_bus_op` that the register model works with and your actual bus sequence item type (`reg2bus`/`bus2reg`). This is what lets the same register model be reused across different bus protocols by simply swapping the adapter, without changing the register/field definitions.

What causes a 'virtual interface is null' error, and how do you debug it?

It means `uvm_config_db#(virtual my_if)::get()` returned without finding a matching `set()` call -- usually caused by a mismatched string path/field name, a type mismatch (e.g., missing modport in the type parameter), or the `set()` happening after the `get()` due to build-order/hierarchy mistakes. Debug by using `+UVM_CONFIG_DB_TRACE` (or the equivalent `set_tracing` API) to print every set/get call and compare the exact path and field name used on both sides.

Why should every component that must complete before the test ends raise its own objection?

If only the top-level test/sequence raises an objection, `run_phase` can end as soon as that one sequence finishes, even if other components (like a scoreboard waiting on in-flight transactions) still have pending work. Having each component that needs to guarantee completion raise its own objection ensures the phase won't end until every dependent piece of work has actually finished.

What is the difference between `uvm_info`, `uvm_warning`, `uvm_error`, and `uvm_fatal`?

They're UVM's severity-graded reporting macros: `uvm_info` is informational (filtered by verbosity level), `uvm_warning` flags a non-fatal issue, `uvm_error` flags a real bug and increments the error count (which typically fails the test in regression), and `uvm_fatal` immediately terminates the simulation -- reserved for conditions where continuing would produce meaningless or misleading further results (e.g., a null virtual interface).

What's the difference between `set_auto_predict` and an explicit `uvm_reg_predictor` in the register model?

`set_auto_predict(1)` updates RAL's internal mirror automatically after every access the register model itself initiates. An explicit `uvm_reg_predictor`, fed from the bus monitor's analysis port, updates the mirror based on any observed bus traffic -- including register accesses RAL didn't initiate (e.g., from another master, or a backdoor write reflected on the bus) -- which auto-predict alone cannot capture.

SVA

What's the difference between an immediate assertion and a concurrent assertion?

An immediate assertion is a procedural statement (`assert (cond) else ...;`) evaluated once, instantly, like an `if` statement -- used for combinational checks inside procedural code. A concurrent assertion (`assert property (...)`) is clocked and can describe temporal behavior spanning multiple cycles using sequences and properties, and runs independently of any single procedural block.

What is the difference between `|->` and `|=>` in a property?

`|->` is overlapping implication: if the antecedent holds in a given cycle, the consequent must hold starting that same cycle. `|=>` is non-overlapping implication: the consequent must hold starting one cycle later. Using `|->` when you mean 'the next cycle' is a common off-by-one bug.

What does `disable iff` protect against, specifically?

It suppresses assertion evaluation whenever its condition is true -- almost always active-low reset (`disable iff (!rst_n)`). Without it, an assertion will evaluate (and likely fail) during reset, when signals are X or in a not-yet-meaningful state, producing false failures unrelated to actual functional behavior.

What's the difference between `$rose(sig)` and comparing `sig` against `$past(sig)`?

``$rose(sig)`` only examines bit 0 of ``sig``, comparing it to the previous sampled value -- fine for single-bit signals, silently wrong for multi-bit values/enums where you actually care about the whole value changing. For multi-bit edge detection (e.g., an FSM leaving IDLE), use an explicit comparison like ``($past(state) == IDLE) && (state != IDLE)`` instead.

What does ``throughout`` mean in a sequence?

``A throughout (seq)`` requires the boolean expression ``A`` to hold true for every cycle spanned by the sequence on the right, from its first cycle to its last. It's the standard idiom for 'this condition must not glitch for the full duration of this window,' such as ``req throughout (##[1:4] ack)``.

How would you write an assertion checking that a signal is one-hot?

Use the built-in ``$onehot(sig)`` system function inside a property: ``assert property (@(posedge clk) disable iff (!rst_n) $onehot(gnt_bus));``. Use ``$onehot0(sig)`` instead if zero-hot (all bits low) should also be considered legal, which ``$onehot`` alone does not allow.

Why should assertions and properties always be labeled?

An unlabeled ``assert property (...)`` still works, but failures are much harder to trace in waveform viewers, regression logs, and coverage-of-assertion reports, since tools generate an anonymous internal name instead of a meaningful one. Labeling (``a_req_ack: assert property (...)``) costs nothing and is the first thing you'll want when triaging a failure.

What is the practical difference between `$stable(sig)` and `$changed(sig)`?

They're exact logical opposites: ``$stable(sig)`` is true when the value is unchanged from the previous sampled clock edge; ``$changed(sig)`` is true when it differs. Which one you reach for is purely about which reads more naturally for the property you're writing -- e.g., 'address must be `$stable` during setup' vs 'a new request is any cycle where `req $changed` to 1' (though the latter is more commonly written with ``$rose``).

How do you check that two events never happen on the same clock cycle (mutual exclusion)?

``assert property (@(posedge clk) disable iff (!rst_n) not (event_a && event_b));`` -- the ``not`` operator combined with a simple AND is the standard way to express 'these two conditions must never both be true simultaneously,' such as a FIFO never seeing both a write-while-full and a read-while-empty in the same cycle.

What's the risk of writing an assertion with an unbounded delay range like `##[1:$]`?

`##[1:$]` means 'eventually, with no upper bound' -- the assertion thread stays alive indefinitely waiting for the consequent, which can silently mask a real bug (a consequent that never arrives may never actually be flagged as a failure within the simulation's runtime) and can also accumulate a large number of pending assertion threads if the antecedent fires repeatedly. In practice, prefer a bounded range (`##[1:N]`) tied to a real specification timeout whenever one exists.

Coverage

What's the difference between code coverage and functional coverage?

Code coverage is automatically derived from the RTL's structure (statement, branch, toggle, FSM coverage) and tells you what RTL was exercised, with no concept of whether the values or scenarios were meaningful. Functional coverage is hand-written by the verification engineer (``covergroup`/`coverpoint`/`cross``) against the verification plan, and tells you whether specific, meaningful scenarios were actually tested.

What does a cross coverage point measure that individual coverpoints don't?

A ``cross`` measures combinations of two or more coverpoints together -- e.g., not just 'did I see every transfer size' and 'did I see both read and write' independently, but specifically whether every size-and-direction combination was exercised. This is usually where real coverage holes are found, since a

constrained-random generator can hit each coverpoint's values individually while never combining them in a particular way.

What's the difference between rand and randc for driving coverage closure?

``rand`` produces repeatable random values with repeats allowed, which can leave some bins under-hit purely by chance in a short run. ``randc`` guarantees every value in its range appears exactly once before any repeats, which can help hit small, bounded coverage spaces (like a 2-bit channel select) faster -- but ``randc`` doesn't scale to wide fields, so it's not a general solution for closing broad coverage.

What does option.per_instance do on a covergroup, and why does it matter?

It controls whether multiple instances of the same covergroup (e.g., one instantiated per agent in a multi-master environment) report coverage separately or get merged into one cumulative result. Without ``per_instance = 1``, you can lose visibility into which specific instance has a coverage hole, since the merged result may show 100% even if one instance never hit a bin that another instance covered.

How do illegal_bins and ignore_bins differ in a coverpoint?

``illegal_bins`` marks certain values as errors -- if one is ever sampled, the simulator raises a runtime error, since hitting that value indicates a bug (e.g., an FSM encoding that should be unreachable). ``ignore_bins`` marks values as simply excluded from coverage accounting, with no error raised -- used for legal-but-uninteresting values you don't want diluting your coverage percentage.

Why is 100% code coverage not sufficient to declare verification complete?

Code coverage only proves that lines/branches/toggles were executed at some point -- it has no concept of whether the specific conditions or corner cases that matter were exercised, or whether the resulting behavior was checked for correctness. A design can reach 100% statement and branch coverage from a handful of simple directed tests that never trigger the interesting corner cases where real bugs hide.

What is the purpose of a coverage collector component (e.g., extending uvm_subscriber) in a UVM environment?

It subscribes to a monitor's analysis port (via ``write()``) so that every observed transaction is sampled into one or more covergroups exactly once, decoupled from the driver, sequencer, and scoreboard. This keeps functional coverage collection passive and reusable -- it observes bus activity without affecting stimulus generation or checking.

What's the difference between sampling a covergroup on a clock edge vs sampling it explicitly with .sample()?

Sampling on a clock edge (``covergroup cg @(posedge clk);``) records a sample every single clock cycle, which can over-count and blur per-transaction meaning. Explicit `.sample()`` calls, typically triggered once per completed transaction from a coverage collector, give you exactly one sample per meaningful event, which is the more common pattern in transaction-level UVM environments.

How would you use bins with ranges to avoid exploding a large address field into millions of bins?

Explicitly define a small number of range bins instead of letting the tool auto-bin every value: e.g., ``bins low = {[32'h0:32'hFFF]}; bins mid = {[32'h1000:32'hFFFF]}; bins high = {[32'h10000:$]};``. This groups the address space into a handful of meaningful regions instead of attempting (and effectively failing) to track coverage for every one of 2^{32} possible values.

Can a design have 100% functional coverage and still ship with a bug? Why?

Yes -- functional coverage is only as good as the verification plan that generated it. If the plan never identified a particular scenario (e.g., simultaneous read/write on an empty FIFO) as something to cover, no coverpoint will ever exist for it, and 100% coverage against an incomplete plan says nothing about that untested scenario. This is why coverage closure is a proxy for 'we tested what we planned to test,' not an absolute guarantee of correctness.

Want the deeper, structured version?

"Roadmap to Verification Engineer" is five focused docs -- SystemVerilog, SVA, UVM, EDA Tools, and Python for VLSI -- for engineers who want a study path instead of scattered blog posts. \$10 each, or \$49 for all five.

<https://divyatejareddy.gumroad.com//ttcedz>

More free tools:

UVM Testbench Skeleton Generator -- <https://dv-verification-hub.vercel.app/tools/uvm-testbench-generator>

SVA Snippet Builder -- <https://dv-verification-hub.vercel.app/tools/sva-snippet-builder>

Full Interview Question Bank -- <https://dv-verification-hub.vercel.app/tools/interview-questions>

All Articles -- <https://dv-verification-hub.vercel.app/articles>